

Init Containers e Sidecar Containers

Obiettivi di Apprendimento

Al termine di questa sezione il corsista sarà in grado di:

- Spiegare il ciclo di vita degli init containers e perché vengono eseguiti prima dei container principali
 - Implementare il pattern init container per gestire dipendenze di avvio (wait-for-DB, caricamento configurazione)
 - Descrivere il pattern sidecar e distinguerlo dagli init containers per ciclo di vita e scopo
 - Configurare un sidecar per raccolta centralizzata di log (Fluentd) e per proxy di rete (Envoy/Istio)
 - Leggere i log di un container specifico in un Pod multi-container con `oc logs -c`
 - Identificare il supporto nativo per sidecar containers in OCP 4.14+ (Kubernetes 1.29)
-

Teoria e Concetti Chiave

Pod Multi-Container: Panoramica

Un Pod può contenere più container che condividono:

- **Rete:** stesso indirizzo IP e stesso namespace di rete (comunicano via `localhost`)
- **Volumi:** possono montare gli stessi volumi
- **Ciclo di vita:** vengono avviati e terminati insieme come unità

OpenShift supporta tre tipologie di container in un Pod:

1. **Init containers** (`spec.initContainers`) — eseguono e completano **prima** che partano i container principali
 2. **Container principali** (`spec.containers`) — il workload applicativo principale
 3. **Sidecar containers nativi** (`spec.initContainers` con `restartPolicy: Always`) — OCP 4.14+/Kubernetes 1.29+
-

Init Containers

Un **init container** è un container speciale che:

1. Viene eseguito **prima** di qualsiasi container principale
2. Deve **completare con successo** (exit code 0) prima che il container successivo parta
3. Se fallisce, viene riavviato secondo la `restartPolicy` del Pod
4. Può avere immagine diversa dai container principali (es. `curl`, `nc`, strumenti admin)

Caratteristiche chiave:

- Esecuzione **sequenziale**: il secondo init container parte solo dopo che il primo è completato
- Ogni init container è **atomico**: parte, fa il suo lavoro, termina
- Condividono gli stessi `volumes` del Pod (utile per scrivere config che il container principale leggerà)
- Hanno separazione di privilegi: possono avere permessi diversi dai container principali

Casi d'uso degli Init Containers

Caso d'uso	Descrizione
Wait-for-service	Attende che un database o un'API sia pronta prima di avviare l'app principale
Pre-caricamento configurazione	Scarica config da un URL o da un Secret e la scrive su un volume condiviso
Migrazione DB schema	Esegue migrate up prima che l'app principale parta
Clone repository	Scarica il codice sorgente su un volume condiviso
Verifica permessi/risorse	Controlla che le risorse necessarie (PVC, Secret) esistano e siano accessibili

Sidecar Containers (Pattern)

Un **sidecar** è un container che gira **accanto** al container principale per tutta la durata del Pod, aggiungendo funzionalità trasversali senza modificare il container principale.

```
Pod
├── Container principale: webapp (porta 8080) → logica applicativa
└── Sidecar: fluentd → raccoglie e spedisce i log
```

Differenze rispetto agli init containers:

Caratteristica	Init Container	Sidecar Container
Ciclo di vita	Completa e termina PRIMA che l'app parta	Gira DURANTE tutta la vita dell'app
Scopo	Preparazione/setup	Funzionalità trasversali in corso
Avvio	Sequenziale, prima dell'app	Contemporaneo all'app
Riavvio	Secondo la restartPolicy del Pod se fallisce	Dovrebbe sempre girare

Casi d'uso dei Sidecar Containers

Caso d'uso	Esempio
Raccolta log	Fluentd raccoglie log da /var/log/app/ e li invia a Elasticsearch/Splunk
Proxy sidecar	Envoy/Istio intercetta il traffico di rete in-/out-bound per mTLS, observability
TLS termination	nginx sidecar gestisce HTTPS, passa HTTP al container principale
Adapter di configurazione	Sidecar che sincronizza la config da ConfigMap/Vault al filesystem locale
Health check esterno	Sidecar che monitora l'app e aggiorna un file di stato

Sidecar Nativi in OCP 4.14+ (Kubernetes 1.29)

Prima di Kubernetes 1.29, i sidecar erano semplicemente container normali in `spec.containers`. Il problema era che il sidecar poteva **bloccare la terminazione del Pod** (se non terminava dopo il container principale) o essere **non disponibile durante l'esecuzione degli init containers**.

Da OCP 4.14+ (Kubernetes 1.29+) è possibile definire un sidecar nativo usando `initContainers` con `restartPolicy: Always`:

```
initContainers:
- name: fluentd-sidecar
  image: ...
  restartPolicy: Always # ← questo lo rende un sidecar nativo
```

Vantaggi del sidecar nativo:

- Parte durante la fase di init (prima dei container principali) ed è disponibile per loro
- Si ferma **dopo** i container principali durante la terminazione del Pod
- Ha un `readinessProbe` che blocca la progression degli init container successivi

 **Suggerimento:** Il sidecar nativo è particolarmente utile per proxy (Istio/Envoy) che devono essere pronti prima che l'applicazione inizi a ricevere traffico.

Condivisione di Risorse tra Containers in un Pod

I container in un Pod possono condividere:

Volumi (emptyDir):

```
volumes:
- name: shared-logs
  emptyDir: {} # Esiste finché il Pod esiste; inizialmente vuoto

# Init container scrive su /init-data
initContainers:
- name: init-config
  volumeMounts:
  - name: shared-logs
    mountPath: /init-data

# Container principale legge da /app/data
containers:
- name: app
  volumeMounts:
  - name: shared-logs
    mountPath: /app/data
```

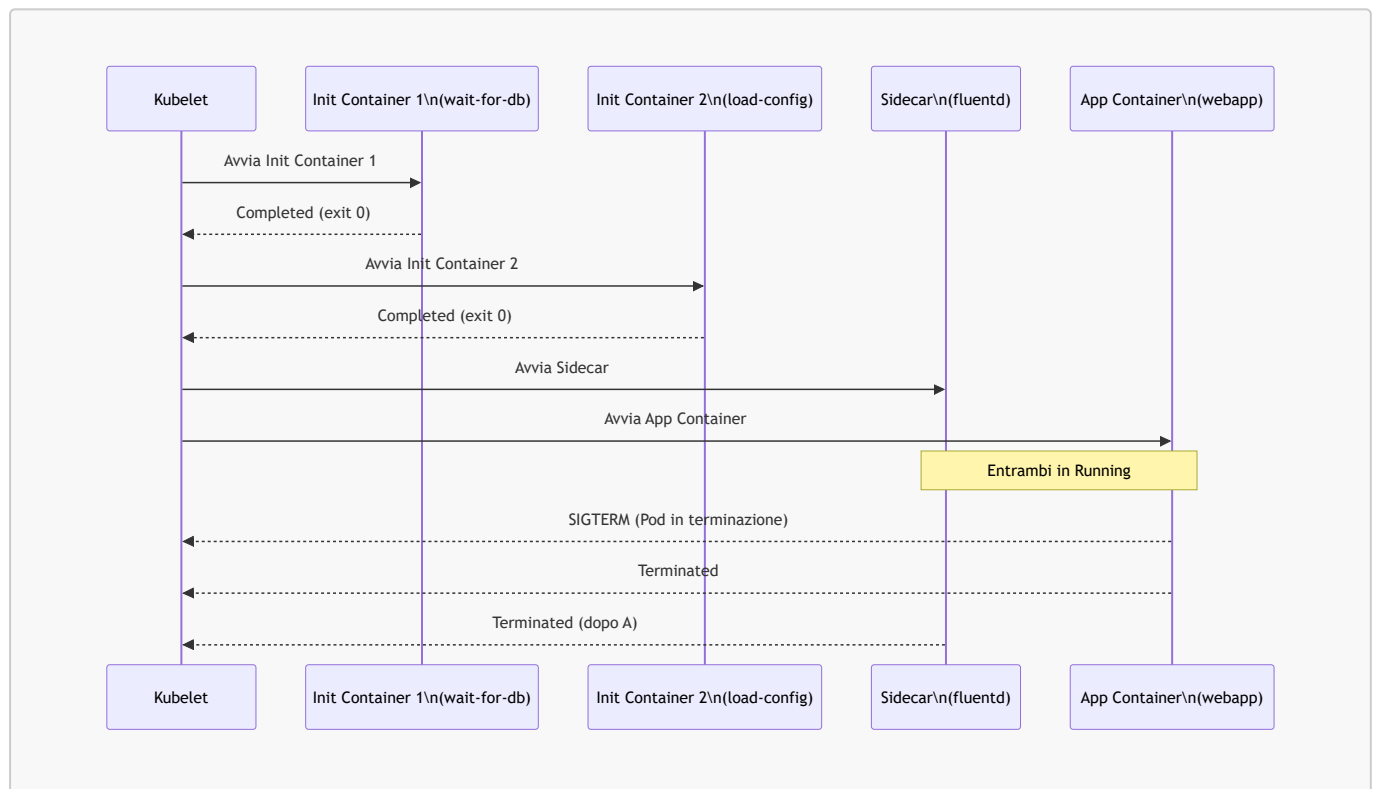
Rete (localhost):

```
# Sidecar espone metriche su porta 9090
containers:
- name: metrics-sidecar
  ports:
  - containerPort: 9090

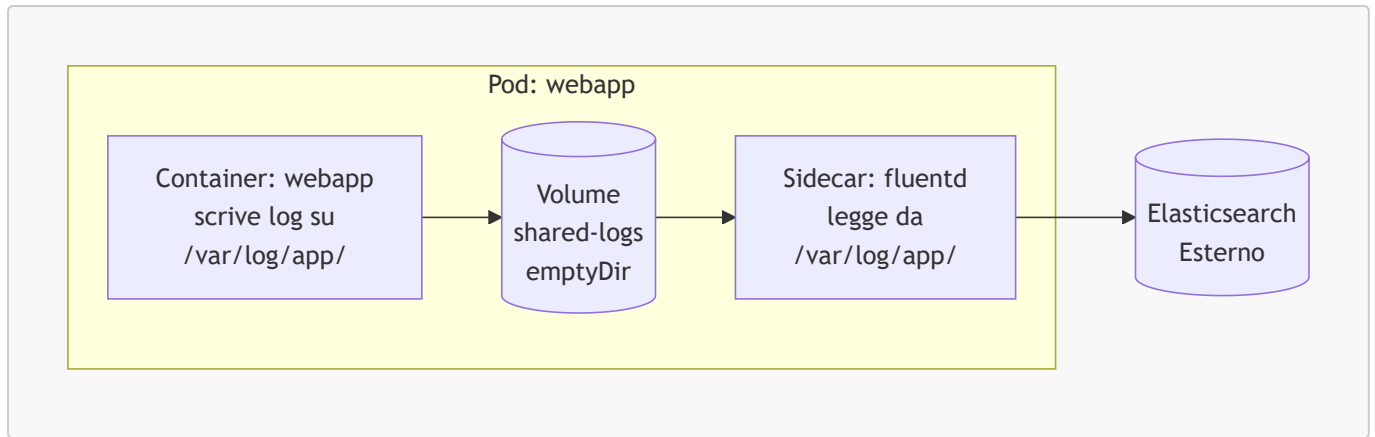
# Il container principale può accedere alle metriche via localhost:9090
- name: app
  env:
  - name: METRICS_URL
    value: "http://localhost:9090/metrics"
```

Diagrammi

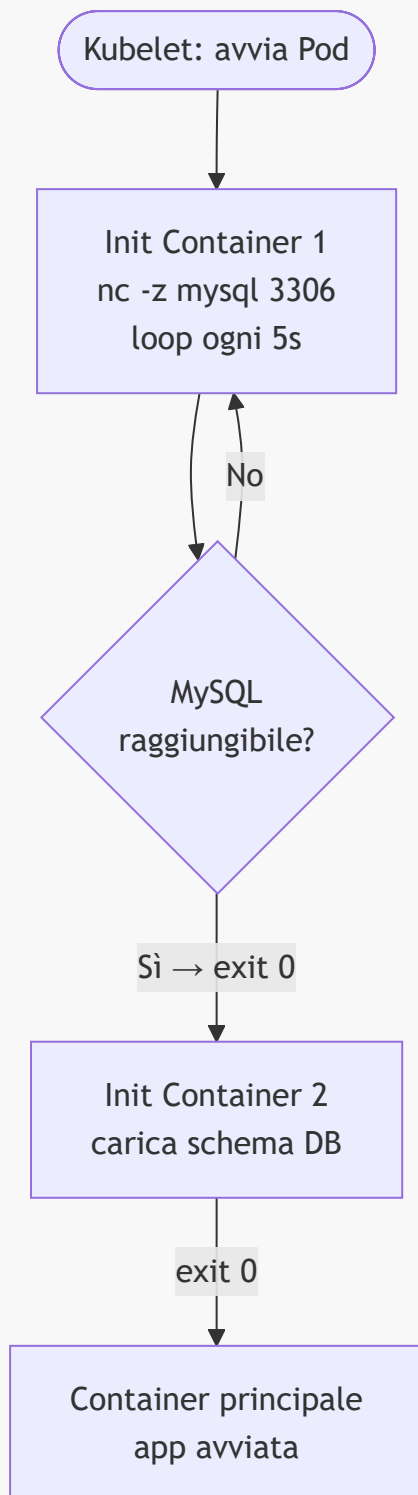
Ordine di Avvio di un Pod Multi-Container



Pattern Sidecar per Logging



Pattern Init Container per Wait-for-Service



Comandi Pratici con Output Atteso

```
# Visualizzare lo stato di tutti i container di un Pod multi-container  
oc get pod webapp-multi -o wide
```

```
# Output atteso:
```

```
# NAME                READY   STATUS    RESTARTS   AGE
```

```
# webapp-multi 2/2 Running 0 5m
# (2/2 = 2 container pronti su 2 container totali, esclusi gli init containers)
```

```
# Vedere lo stato degli init containers di un Pod
oc describe pod webapp-multi | grep -A 20 "Init Containers:"
```

```
# Output atteso:
# Init Containers:
#   wait-for-db:
#     State:      Terminated
#     Reason:     Completed
#     Exit Code:  0
#     Ready:      True
#   load-config:
#     State:      Terminated
#     Reason:     Completed
#     Exit Code:  0
#     Ready:      True
```

```
# Leggere i log di un container specifico (init container o sidecar)
oc logs webapp-multi -c wait-for-db
```

```
# Output atteso:
# Waiting for MySQL at mysql-headless:3306...
# MySQL not ready yet, retrying in 5s...
# MySQL is ready! Proceeding.
```

```
# Leggere i log del container principale
oc logs webapp-multi -c webapp
```

```
# Output atteso:
# [INFO] Starting webapp on port 8080
# [INFO] Connected to database
# [INFO] Server ready
```

```
# Leggere i log del sidecar in tempo reale
oc logs webapp-multi -c fluentd -f
```

```
# Output atteso:
# [info]: Reading config file /fluentd/etc/fluent.conf
# [info]: Starting fluentd worker pid=1 ...
# [info]: fluentd worker is now running worker=0
```

```
# Aprire una shell nel container specifico di un Pod multi-container
oc exec -it webapp-multi -c webapp -- /bin/bash
```

```
# Output atteso:
# bash-5.1$
```

```
# Accedere al sidecar (es. per debug del proxy)
oc exec -it webapp-multi -c fluentd -- sh
```

```
# Visualizzare tutti i container del Pod (inclusi init) con il loro stato
oc get pod webapp-multi -o jsonpath='{range .status.initContainerStatuses[*]}
{name}: {state}{"\n"}{end}'
```

```
# Output atteso:
# wait-for-db: map[terminated:map[exitCode:0 reason:Completed]]
# load-config: map[terminated:map[exitCode:0 reason:Completed]]
```

```
# Ottenere lo stato di tutti i container (init + principali + sidecar) in formato
tabella
oc get pod webapp-multi -o json | \
python3 -c "import json,sys; d=json.load(sys.stdin); \
[print(f'{c[\"name\"]}: {list(c[\"state\"]').keys()[0]}') \
for c in d['status'].get('containerStatuses', [])]"
```

Esempi YAML Completi

Pod con Init Containers per Wait-for-DB e Load-Config

```
apiVersion: v1
kind: Pod
metadata:
  name: webapp-with-init
  namespace: myproject
  labels:
    app: webapp
spec:
  restartPolicy: Always
  # Volume condiviso tra init containers e container principale
  volumes:
  - name: app-config
    emptyDir: {}

  initContainers:
  # Init Container 1: attende che MySQL sia pronto prima di procedere
```



```
- name: wait-for-db
  image: registry.access.redhat.com/ubi9/ubi-minimal:9.4
  command: ["/bin/bash", "-c"]
  args:
  - |
    echo "Waiting for MySQL to be ready..."
    until nc -z mysql-headless.myproject.svc.cluster.local 3306; do
      echo "MySQL not ready yet, retrying in 5s..."
      sleep 5
    done
    echo "MySQL is ready! Proceeding."
  resources:
    requests:
      memory: "32Mi"
      cpu: "10m"
    limits:
      memory: "64Mi"
      cpu: "50m"

# Init Container 2: scarica la configurazione e la scrive su volume condiviso
- name: load-config
  image: registry.access.redhat.com/ubi9/ubi-minimal:9.4
  command: ["/bin/bash", "-c"]
  args:
  - |
    echo "Loading application configuration..."
    # Copia la configurazione dal ConfigMap nel volume condiviso
    cp /tmp/app.properties /config/app.properties
    echo "Configuration loaded."
  volumeMounts:
  - name: app-config
    mountPath: /config
  resources:
    requests:
      memory: "32Mi"
      cpu: "10m"
    limits:
      memory: "64Mi"
      cpu: "50m"

containers:
# Container principale: legge la configurazione dal volume condiviso
- name: webapp
  image: registry.access.redhat.com/ubi9/ubi-minimal:9.4
  ports:
  - containerPort: 8080
  volumeMounts:
  # Legge la configurazione scritta dall'init container
  - name: app-config
    mountPath: /app/config
    readOnly: true
  resources:
    requests:
      memory: "128Mi"
```

```

    cpu: "100m"
  limits:
    memory: "256Mi"
    cpu: "500m"
  readinessProbe:
    httpGet:
      path: /ready
      port: 8080
    initialDelaySeconds: 10
    periodSeconds: 5

```

Pod con Sidecar per Logging (Fluentd)

```

apiVersion: v1
kind: Pod
metadata:
  name: webapp-with-sidecar
  namespace: myproject
  labels:
    app: webapp
spec:
  restartPolicy: Always
  # Volume condiviso per i log tra webapp e fluentd
  volumes:
  - name: shared-logs
    emptyDir: {}

  containers:
  # Container principale: scrive log su file nel volume condiviso
  - name: webapp
    image: registry.access.redhat.com/ubi9/ubi-minimal:9.4
    command: ["/bin/bash", "-c"]
    args:
    - |
      while true; do
        echo "[$(date -u +%Y-%m-%dT%H:%M:%SZ)] INFO Request processed" >>
/var/log/app/app.log
        sleep 10
      done
    volumeMounts:
    # Scrive i log nella directory condivisa con il sidecar
    - name: shared-logs
      mountPath: /var/log/app
  resources:
    requests:
      memory: "64Mi"
      cpu: "50m"
    limits:
      memory: "128Mi"
      cpu: "200m"

```

```
# Sidecar: legge i log scritti da webapp e li spedisce a Elasticsearch
- name: fluentd
  image: registry.redhat.io/openshift-logging/fluentd-rhel9:latest
  volumeMounts:
    # Legge i log dalla directory condivisa con webapp
    - name: shared-logs
      mountPath: /var/log/app
      readOnly: true
  resources:
    requests:
      memory: "100Mi"
      cpu: "50m"
    limits:
      memory: "200Mi"
      cpu: "200m"
  env:
    - name: ELASTICSEARCH_HOST
      value: "elasticsearch.logging.svc.cluster.local"
    - name: ELASTICSEARCH_PORT
      value: "9200"
```

Pod con Sidecar Nativo OCP 4.14+ (initContainer con restartPolicy: Always)

```
apiVersion: v1
kind: Pod
metadata:
  name: webapp-native-sidecar
  namespace: myproject
spec:
  restartPolicy: Always
  volumes:
    - name: shared-logs
      emptyDir: {}

  initContainers:
    # Sidecar nativo: definito in initContainers ma con restartPolicy: Always
    # → parte prima dell'app e continua a girare per tutta la vita del Pod
    - name: fluentd-sidecar
      image: registry.redhat.io/openshift-logging/fluentd-rhel9:latest
      restartPolicy: Always # ← questa direttiva rende il container un "sidecar nativo"
      volumeMounts:
        - name: shared-logs
          mountPath: /var/log/app
          readOnly: true
      resources:
        requests:
          memory: "100Mi"
          cpu: "50m"
        limits:
          memory: "200Mi"
```

```

    cpu: "200m"
# La readinessProbe del sidecar nativo blocca l'avvio dei container successivi
readinessProbe:
  exec:
    command: ["pgrep", "fluentd"]
  initialDelaySeconds: 5
  periodSeconds: 5

containers:
- name: webapp
  image: registry.access.redhat.com/ubi9/ubi-minimal:9.4
  command: ["/bin/bash", "-c"]
  args: ["while true; do echo LOG; sleep 5; done >> /var/log/app/app.log"]
  volumeMounts:
  - name: shared-logs
    mountPath: /var/log/app
  resources:
    requests:
      memory: "64Mi"
      cpu: "50m"
    limits:
      memory: "128Mi"
      cpu: "200m"

```

Note Specifiche per Azure ARO

Init Containers e SCC su ARO

Su ARO, gli init containers sono soggetti alle stesse SCC dei container principali. Se un init container deve eseguire operazioni privilegiate (es. `chown` su un volume montato), potrebbe necessitare di SCC elevate:

```

# Verificare la SCC richiesta dall'init container
oc adm policy scc-subject-review -f pod-with-init.yaml

# Se necessario, concedere SCC al ServiceAccount del Pod
oc adm policy add-scc-to-serviceaccount anyuid -z myapp-sa -n myproject

```

⚠ Attenzione: Su ARO, evita di usare `privileged` SCC per gli init containers. Per operazioni di `chown/chmod` su volumi, preferisci `securityContext.fsGroup` a livello di Pod spec.

Sidecar e Istio Service Mesh su ARO

Se il cluster ARO ha OpenShift Service Mesh (Istio) abilitato, viene iniettato automaticamente un sidecar Envoy in ogni Pod del namespace abilitato. Per verificare:

```

# Abilitare l'iniezione automatica del sidecar Istio nel namespace
oc label namespace myproject istio-injection=enabled

```

```
# Verificare che il sidecar sia stato iniettato in un Pod
oc get pod webapp-multi -o jsonpath='{.spec.containers[*].name}'
# Output atteso: webapp istio-proxy
```

 **Suggerimento:** Con Service Mesh, il container **istio-proxy** viene iniettato automaticamente come sidecar che gestisce mTLS, tracing e traffic management. Non è necessario configurarlo manualmente nei YAML applicativi.

Risorse per Init Containers su ARO

Le risorse degli init containers contribuiscono al calcolo dei limiti del Pod. Su ARO con ResourceQuota strette, assicurarsi che i **resources.requests** degli init containers siano coerenti con la quota disponibile:

```
# Verificare le risorse "effettive" richieste dal Pod (init + containers)
oc describe pod webapp-with-init | grep -A 10 "QoS Class:"

# Le risorse del Pod considerano il massimo tra:
# - Somma risorse container principali
# - Risorsa più alta singolo init container
```

Riepilogo dei Punti Chiave

- Gli init containers vengono eseguiti in ordine sequenziale **prima** dei container principali e devono completare con successo (exit 0)
- Il volume **emptyDir** è il meccanismo standard per condividere dati tra init container e container principale
- I sidecar containers girano **in parallelo** al container principale per l'intera durata del Pod, aggiungendo funzionalità trasversali
- In OCP 4.14+ (Kubernetes 1.29+), i sidecar nativi si definiscono in **initContainers** con **restartPolicy: Always**
- **oc logs -c <container-name>** è il comando per leggere i log di un container specifico in un Pod multi-container
- Il pattern init container + sidecar è la base dell'architettura Service Mesh (Istio inietta automaticamente il sidecar Envoy)
- Su ARO verificare sempre le SCC per init containers che eseguono operazioni privilegiate

Domande di Verifica

1. Un Pod ha due init containers: **init-a** e **init-b**. In quale ordine vengono eseguiti?

- A) In parallelo: entrambi partono contemporaneamente
- B) **init-b** prima, poi **init-a**
- C) **init-a** prima, poi **init-b** solo dopo che **init-a** è completato con successo 
- D) In ordine casuale

2. Qual è la differenza principale tra un sidecar container e un init container?

- A) Un init container ha più risorse di un sidecar
- B) Un sidecar gira in parallelo al container principale per tutta la vita del Pod; l'init container completa prima che l'app parta ☒
- C) Un init container può accedere ai volumi del Pod; il sidecar no
- D) Non c'è differenza: entrambi completano prima che l'app parta

3. Un init container usa `nc -z database 5432` in un loop. Quando termina?

- A) Dopo un timeout fisso di 30 secondi
- B) Solo se la porta 5432 del database è raggiungibile (exit 0) ☒
- C) Immediatamente, indipendentemente dallo stato del database
- D) Quando la `livenessProbe` del container principale è configurata

4. Quale comando permette di leggere i log del sidecar `fluentd` in un Pod chiamato `webapp-multi`?

- A) `oc logs webapp-multi --sidecar fluentd`
- B) `oc logs fluentd --pod webapp-multi`
- C) `oc logs webapp-multi -c fluentd` ☒
- D) `oc describe pod webapp-multi | grep fluentd`

5. In OCP 4.14+, come si definisce un sidecar nativo in un Pod?

- A) Usando il campo `spec.sidecars` introdotto in Kubernetes 1.29
- B) Aggiungendo l'annotazione `sidecar.io/inject: "true"` al Pod
- C) Definendo il container in `spec.initContainers` con `restartPolicy: Always` ☒
- D) Non è possibile: i sidecar nativi non sono supportati in OCP 4.14